



Studiengang: Informations- und Kommunikationstechnik

Sensortechnik

Inkrement 2: Betriebszustandsmonitoring

Vorgelegt von: Beruk Chimi, 2726568
Daniel Jakovlev, 2314952
Abgabedatum: 29.1.2024
Dozent: Prof. Dr.-Ing. Dipl. -Kfm. Jörg Dahlkemper

Inhaltsverzeichnis

1	Einleitung	3
2	Vorbereitung	3
2.1	ESP32 Mikrocontroller	3
2.2	MPU 6050 Sensor	3
2.3	Signalverarbeitung und Merkmalsextraktion	3
2.4	K-Nearest-Neighbor-Klassifikator (kNN)	4
2.5	Parameter zur Korrelationsanalyse von Merkmalen	6
3	Durchführung	7
4	Ergebnisse	9
4.1	Vergleich der Klassifikationsgenauigkeit	9
4.2	Auswertung der Konfusionsmatrizen	10
4.3	Schlussfolgerung	10

Abbildungsverzeichnis

1	schaltung	9
2	Darstellung der Konfusionsmatrix mit Berechnung der Merkmale.	11
3	Darstellung der Konfusionsmatrix der Rohdaten ohne Merkmalsberechnung.	11
4	Darstellung des Histogramms mit Merkmalen und Rohdaten.	12
5	Korrelationmatrix der Merkmalen und Rohdaten.	13

Tabellenverzeichnis

1	Pinbelegung	9
---	-----------------------	---

1 Einleitung

Im Rahmen dieses Laborversuchs soll ein Betriebszustandsmonitoring für eine Fernbedienung entwickelt werden. Ziel ist es, mithilfe des ESP32-Mikrocontroller-Boards und des Bewegungssensors MPU 6050 die aktuellen Zustände der Fernbedienung zu erkennen: *Ruhe*, *Transport* oder *aktiver Steuerbetrieb*.

Die Umsetzung umfasst sowohl die Integration der Sensor-Hardware und die Verarbeitung der Rohdaten als auch die Entwicklung einer automatisierten Zustandserkennung. Ein k-Nearest-Neighbor-Klassifikator wird implementiert, um die Zustände auf Grundlage gelabelter Daten präzise zu identifizieren. Für die Funktionsprüfung werden statistische Merkmale aus den Rohdaten extrahiert, und die Ergebnisse werden anhand von Leistungskennzahlen wie der Accuracy und einer Konfusionsmatrix bewertet.

2 Vorbereitung

2.1 ESP32 Mikrocontroller

Der ESP32 ist ein vielseitiges Mikrocontroller-Board. Es arbeitet mit einer Betriebsspannung von 3,3V, wird jedoch über den USB-Port mit 5V versorgt. Der ESP32 unterstützt den I2C-Bus, der für die Kommunikation mit externen Peripheriegeräten wie dem MPU 6050-Sensor verwendet wird. Die Standard-I2C-Pins des ESP32 sind G21 (SDA) und G22 (SCL).

2.2 MPU 6050 Sensor

Die Motion Processing Unit (MPU) 6050 ist ein Sensor zur Erfassung von Bewegungs- und Lageinformationen. Sie kombiniert ein 3-Achsen-Gyroskop und einen 3-Achsen-Beschleunigungssensor, wodurch Neigungs- und Bewegungsdaten erfasst werden können. Über die I2C-Schnittstelle liefert der Sensor Rohdaten in Form von zweierkomplementierten 16-Bit-Werten mit $10Hz$, die in Echtzeit ausgewertet werden können.

2.3 Signalverarbeitung und Merkmalsextraktion

Zur Zustandserkennung werden die vom MPU 6050 bereitgestellten Rohdaten aufbereitet. Die Zeitreihen der Beschleunigungs- und Rotationswerte dienen als Grundlage für die Berechnung statistischer Merkmale wie Mittelwert, Varianz oder Standardabweichung. Diese Merkmale ermöglichen eine robuste Klassifikation der Zustände.

Die Berechnung der Merkmale erfolgt sowohl für die Beschleunigungsdaten ($AccX$, $AccY$, $AccZ$) als auch für die Gyroskopdaten ($GyroX$, $GyroY$, $GyroZ$). Die extrahierten Merkmale umfassen:

- **Mittelwert:** Gibt die durchschnittliche Bewegung über das Fenster an.
- **Standardabweichung:** Beschreibt die Streuung der Daten.
- **Maximum und Minimum:** Erfassen die Extremwerte der Bewegungen.

Die Berechnung der Merkmale für ein Fenster erfolgt wie folgt:

$$\begin{aligned}\text{Mittelwert } (\bar{x}) &= \frac{1}{N} \sum_{i=1}^N x_i, \\ \text{Standardabweichung } (\sigma_x) &= \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}, \\ \text{Maximum } (\max_x) &= \max(x_1, x_2, \dots, x_N), \\ \text{Minimum } (\min_x) &= \min(x_1, x_2, \dots, x_N),\end{aligned}$$

wobei x_i die Sensordatenpunkte eines Fensters und N die Anzahl der Punkte im Fenster sind.

2.4 K-Nearest-Neighbor-Klassifikator (kNN)

Der k-Nearest-Neighbor-Algorithmus ist ein einfaches und effizientes Klassifikationsverfahren. Es basiert darauf, die Ähnlichkeit eines neuen Datenpunkts zu bereits gelabelten Daten zu bestimmen. Der Algorithmus berücksichtigt die k nächsten Nachbarn im Merkmalsraum und entscheidet basierend auf deren Mehrheitsklasse. Für die Zustandserkennung der Fernbedienung wird ein kNN-Modell auf Basis gelabelter Trainingsdaten erstellt und die Leistung anhand von Accuracy und einer Konfusionsmatrix bewertet.

Nach der Merkmalsextraktion werden die Merkmalsvektoren als Eingabe für einen *k-Nearest-Neighbor* (kNN)-Klassifikator verwendet.

Trainingsphase

Die extrahierten Merkmalsvektoren werden in Trainings- und Testdaten unterteilt. Die Trainingsdaten werden verwendet, um den kNN-Klassifikator zu trainieren. Dabei wird der optimale Wert für k mittels Kreuzvalidierung ermittelt.

Klassifikationsphase

Im Klassifikationsschritt berechnet der kNN-Algorithmus die euklidischen Distanzen zwischen dem neuen Merkmalsvektor und allen Trainingsdatenpunkten:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2},$$

wobei x der neue Datenpunkt, y ein Trainingspunkt und n die Anzahl der Merkmale ist. Die Klasse des neuen Punktes wird durch die Mehrheit der nächsten k -Nachbarn bestimmt.

Ergebnisse und Bewertung

Die Leistung des kNN-Klassifikators wird anhand der Accuracy und einer Konfusionsmatrix bewertet. Die Accuracy misst den Anteil korrekt klassifizierter Datenpunkte:

$$\text{Accuracy} = \frac{\text{Anzahl korrekter Vorhersagen}}{\text{Gesamtanzahl der Vorhersagen}}.$$

Die Konfusionsmatrix zeigt die Verteilung der Klassifikationsergebnisse und bietet Einblick in falsch-positive und falsch-negative Entscheidungen.

Diskussion des Parameters k im k-Nearest-Neighbor-Klassifikator

Der Parameter k im k-Nearest-Neighbor (kNN)-Klassifikator gibt die Anzahl der Nachbarn an, die bei der Klassifikation eines Datenpunkts berücksichtigt werden. Die Wahl des k -Wertes beeinflusst die Leistung des Modells erheblich und stellt einen zentralen Aspekt der Modelloptimierung dar.

Einfluss kleiner k -Werte

Ein kleiner k -Wert, wie $k = 1$, führt dazu, dass der Klassifikator ausschließlich auf den nächstgelegenen Nachbarn im Merkmalsraum basiert. Dies kann zu einer hohen Varianz führen, da das Modell empfindlich auf Rauschen in den Daten reagiert. Kleinere Werte von k können zu einem sogenannten *Overfitting* führen, bei dem das Modell die Trainingsdaten sehr genau beschreibt, jedoch eine schlechtere Generalisierungsfähigkeit aufweist.

Einfluss großer k -Werte

Mit steigenden k -Werten wird die Entscheidung für die Klassifikation zunehmend auf Basis einer größeren Nachbarschaft getroffen. Dies reduziert die Varianz, kann jedoch die Bias des Modells erhöhen. Bei sehr großen k -Werten wird die Klassifikation von der Mehrheitsklasse dominiert, was die Entscheidungsgrenze weniger flexibel macht und feine Muster in den Daten ignorieren kann.

Optimale Wahl von k

Die optimale Wahl des Parameters k hängt von der Größe des Datensatzes, der Verteilung der Klassen und der Art der Daten ab. In der vorliegenden Implementierung wurde $k = 20$ gewählt, basierend auf Experimenten mit verschiedenen Werten. Die Wahl dieses k -Wertes bietet eine Balance zwischen Varianz und Bias, wodurch eine robuste Klassifikationsleistung auf den Testdaten erreicht wird.

Zur Bestimmung des optimalen k -Wertes können Methoden wie die Kreuzvalidierung eingesetzt werden. Dabei wird die Klassifikationsleistung (z. B. Accuracy) für verschiedene Werte von k evaluiert. Die Ergebnisse einer solchen Optimierung könnten in Form eines Diagramms, das den Verlauf der Accuracy in Abhängigkeit von k zeigt, dargestellt werden.

Listing 1: Berechnung des optimalen k für kNN

```
for k in range(1,30):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    print(f"Genauigkeit_fuer_k={k}:", knn.score(X_test, y_test))
```

2.5 Parameter zur Korrelationsanalyse von Merkmalen

Bei der Durchführung einer Korrelationsanalyse zur Untersuchung der Beziehungen zwischen Merkmalen sind folgende Parameter relevant: [Abbildung 5](#)

1. Pearson-Korrelationskoeffizient:

Dieser misst den linearen Zusammenhang zwischen zwei kontinuierlichen Variablen. Mit ihm kann festgestellt werden, wie stark einzelne Merkmale (z. B. `mean_x`, `std_x`, etc.) miteinander korreliert sind. Eine hohe Korrelation (nahe 1 oder -1) zwischen zwei Merkmalen deutet darauf hin, dass sie ähnliche Informationen enthalten. In solchen Fällen kann eines der Merkmale entfernt werden, um die Datenmenge zu reduzieren.

2. Heatmap zur Visualisierung:

Eine Korrelationsmatrix, dargestellt als Heatmap, bietet eine übersichtliche Visualisierung der Beziehungen zwischen den Merkmalen. Dadurch können stark korrelierende Parameter und potenziell redundante Merkmale schnell identifiziert werden.

3. Unterscheidung zwischen unabhängigen Achsen:

Bei Sensoranwendungen können Achsen (z. B. `AccX`, `AccY`, `AccZ`) physikalisch zusammenhängen. Zum Beispiel könnten `AccX` und `AccY` gemeinsam eine Bewegungsrichtung beschreiben. Eine Analyse kann zeigen, ob die Merkmale unabhängig genug sind oder ob eine Transformation, wie die Hauptkomponentenanalyse (PCA), sinnvoll wäre.

4. Statistische Signifikanz (p-Wert):

Neben der Stärke der Korrelation ist es wichtig, die statistische Signifikanz zu bewerten. Ein kleiner *p-Wert* (< 0.05) zeigt, dass die Korrelation mit hoher Wahrscheinlichkeit nicht zufällig ist. So wird sichergestellt, dass nur relevante Beziehungen berücksichtigt werden.

5. Feature-Wichtigkeit im Modell:

Nicht alle Merkmale sind für die Klassifikation gleich wichtig. Modelle wie Random Forests oder Gradient Boosting können die Bedeutung einzelner Merkmale berechnen, was dabei hilft, unwichtige Merkmale zu eliminieren, die keinen Mehrwert für das Modell liefern.

3 Durchführung

Für die Durchführung werden die folgenden Materialien benötigt:

- ESP32-Board
- MPU6050
- Computer
- Jumperkabel
- Breadboard

Die folgende Pinbelegung ist zu beachten [Tabelle 1](#):

Nachdem die gesamte Hardware aufgebaut wurde, soll der Arduino-Code auf das ESP32-Board geladen werden. Die [Abbildung 1](#) veranschaulicht die Verbindung zwischen den einzelnen Komponenten.

Echtzeit-Zustandsvorhersage mit kNN-Klassifikator

auf Basis von Sensordaten, die über eine serielle Schnittstelle von einem Arduino empfangen werden. Die Hauptkomponenten des Codes sind die Datenaufnahme, die Merkmalsberechnung, die Zustandsklassifikation und die kontinuierliche Verarbeitung in Echtzeit.

1. Datenaufnahme:

Die Sensordaten (Beschleunigung und Gyroskop) werden kontinuierlich vom angeschlossenen ESP32 empfangen. Eine serielle Verbindung wird mit der Bibliothek `serial` hergestellt, und die empfangenen Daten werden in einem Puffer (`deque`) gespeichert, der Platz für die letzten 120 Messungen bietet.

2. Merkmalsberechnung:

Um die Rohdaten zu analysieren, werden Merkmale wie Mittelwert, Standardabweichung, Minimum und Maximum aus den Beschleunigungs- und Gyroskopdaten berechnet. Diese Berechnung erfolgt in Zeitfenstern. Die Funktion `calculate_features` führt diese Berechnungen durch und gibt einen DataFrame mit einem vordefinierten Merkmalsfenster zurück. Anschließend wird ein k-Nearest-Neighbor-Klassifikator (kNN) mit diesen Merkmalen trainiert, um die Zustände vorherzusagen.

3. Training mit Rohdaten:

Neben der Merkmalsberechnung wird ein zweiter Ansatz verwendet, bei dem das kNN-Modell direkt mit den unmodifizierten Rohdaten (z. B. Beschleunigungs- und Gyroskopwerten) trainiert wird. Die Rohdaten werden auf die gleiche Weise wie die Merkmale skaliert, um sicherzustellen, dass alle Wertebereiche vergleichbar sind. Dieser Ansatz ermöglicht einen direkten Vergleich der Leistung des kNN-Modells mit und ohne Merkmalsberechnung.

4. Zustandsklassifikation:

Die berechneten Merkmale oder die Rohdaten werden an das kNN-Modell übergeben, das die Zustände „Ruhe“, „Transport“ und „Betrieb“ vorhersagen kann. Das Modell wird für beide Varianten separat trainiert, und die Ergebnisse werden miteinander verglichen, um die Effektivität der Merkmalsberechnung zu bewerten.

5. Echtzeitverarbeitung:

Während der kontinuierlichen Datensammlung wird die Merkmalsberechnung ausgeführt, sobald der Puffer ausreichend Daten enthält. Diese Merkmale werden skaliert und an das kNN-Modell übergeben, um den aktuellen Zustand vorherzusagen. Der vorhergesagte Zustand wird in der Konsole ausgegeben.

Code-Hervorhebung:

Im Folgenden wird ein Ausschnitt des Hauptteils des Codes gezeigt, der die Merkmalsberechnung und die Zustandsklassifikation durchführt:

Listing 2: Merkmalsberechnung und Zustandsklassifikation

```
def calculate_features(df):
    features = []
    window_size = 30

    for i in range(0, len(df), window_size):
        window = df.iloc[i:i + window_size]
        mean_x = window['AccX'].mean()
        std_x = window['AccX'].std()
        # Weitere Berechnungen fuer std_y, max_z, etc.
        features.append([mean_x, std_x, ...])

    features_df = pd.DataFrame(features, columns=[
        'mean_x', 'std_x', ...])
    return features_df

def read_serial_data():
    global current_state
    while running:
        line = ser.readline().decode('utf-8').strip()
        values = [float(x) for x in line.split(",")]
        data_queue.append(values)

        if len(data_queue) == 120:
            window = pd.DataFrame(list(data_queue), columns=["AccX", ...])
            features_df = calculate_features(window)
            scaled_features = scaler_loaded.transform(features_df.iloc[:, -30:])
            prediction = knn_loaded.predict(scaled_features)
            current_state = prediction[0]
            print(f"Vorhergesagter_Zustand: {current_state}")
```

Dieser Ansatz kombiniert Sensordatenverarbeitung und maschinelles Lernen, um eine robuste und flexible Echtzeitüberwachung der Betriebszustände zu ermöglichen.

Tabelle 1: Pinbelegung

MPU6050	ESP32-Board
VCC	3v3
GRND	GRND
SDA	G21
SCL	G22

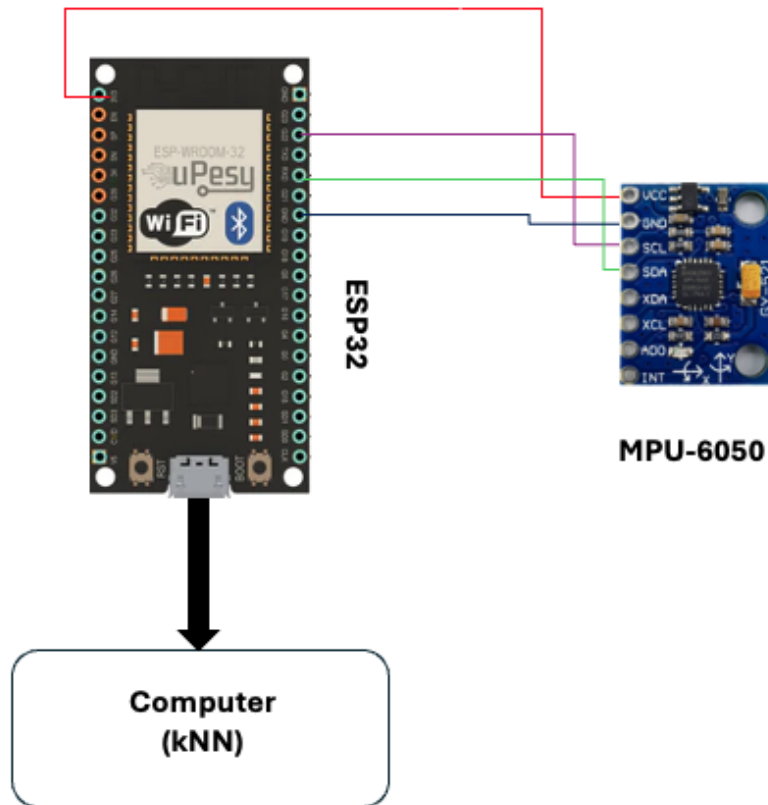


Abbildung 1: Schaltungsaufbau. ESP32 mit dem MPU6050

4 Ergebnisse

4.1 Vergleich der Klassifikationsgenauigkeit

Um die Effektivität der Merkmalsberechnung zu bewerten, wurden zwei Ansätze getestet: der k-Nearest-Neighbor-Klassifikator (kNN) wurde sowohl mit den Rohdaten als auch mit den berechneten Merkmalen trainiert. Die Ergebnisse zeigen deutliche Unterschiede in der Genauigkeit der Vorhersagen.

Beim Training mit den **Rohdaten** erreichte der Klassifikator eine Genauigkeit von **98,03 %**. Dies zeigt, dass kNN in der Lage ist, selbst aus den ungefilterten Sensordaten zuverlässige Vorhersagen zu treffen.

Durch die Verwendung der **berechneten Merkmale** konnte die Genauigkeit jedoch auf **99,44 %** gesteigert werden. Dies unterstreicht die Bedeutung der Merkmalsextraktion, da sie zusätzliche Informationen liefert, die die Trennbarkeit der Zustände verbessern und somit die Performance des Modells optimieren.

Zusammenfassend zeigt der Vergleich, dass die Merkmalsberechnung ein effektives Mittel ist, um die Leistung des Klassifikators signifikant zu erhöhen.

4.2 Auswertung der Konfusionsmatrizen

Die Auswertung der Konfusionsmatrizen zeigt klare Unterschiede in der Modellleistung zwischen der Klassifikation mit Merkmalen und der Klassifikation basierend auf Rohdaten:

- **Mit Merkmalen:**

- Die Konfusionsmatrix mit Merkmalen zeigt eine hohe Präzision und Genauigkeit. Es gibt nur wenige Fehlklassifikationen, was darauf hindeutet, dass die Merkmalsextraktion dazu beiträgt, relevante Informationen aus den Rohdaten zu extrahieren und die Unterscheidung zwischen den Klassen zu verbessern.
- Die meisten Einträge befinden sich entlang der Hauptdiagonale der Matrix, was auf eine hohe Übereinstimmung zwischen den tatsächlichen und vorhergesagten Klassen hinweist.
- Dies deutet darauf hin, dass die Modellleistung durch die Merkmalsextraktion optimiert wurde, indem irrelevante oder verrauschte Informationen reduziert wurden [Abbildung 2](#).

- **Mit Rohdaten:**

- Die Konfusionsmatrix basierend auf den Rohdaten zeigt eine etwas geringere Genauigkeit im Vergleich zur Matrix mit Merkmalen. Es gibt mehr Einträge außerhalb der Hauptdiagonale, was auf eine höhere Anzahl an Fehlklassifikationen hinweist.
- Die höhere Fehlerrate könnte auf die Schwierigkeit des Modells hinweisen, rohe Sensordaten direkt zu interpretieren, da keine Vorverarbeitung oder Transformation der Daten erfolgt ist.
- Ohne Merkmale ist das Modell stärker anfällig für Rauschen oder irrelevante Schwankungen in den Sensordaten, was die Trennbarkeit der Klassen beeinträchtigt [Abbildung 3](#).

- **Vergleich:**

- Die Klassifikation mit Merkmalen liefert konsistent bessere Ergebnisse als die Klassifikation mit Rohdaten. Dies unterstreicht die Bedeutung der Merkmalsextraktion, um aus den Rohdaten kompakte, aussagekräftige Informationen zu generieren.
- Die Unterschiede in der Leistung sind insbesondere bei Klassen mit ähnlichen Eigenschaften deutlich: Das Modell mit Merkmalen zeigt hier eine klarere Trennung, während das Modell mit Rohdaten häufiger Fehler macht.

4.3 Schlussfolgerung

Die Analyse der Konfusionsmatrizen bestätigt, dass die Merkmalsextraktion einen entscheidenden Einfluss auf die Modellleistung hat. Sie reduziert die Fehlklassifikationen und erhöht die Genauigkeit, indem sie irrelevante Informationen eliminiert und die Diskriminationsfähigkeit des Modells verbessert. Die Ergebnisse zeigen, dass die Verwendung von Merkmalen bei der Analyse und Klassifikation von Sensordaten einen klaren Vorteil bietet.

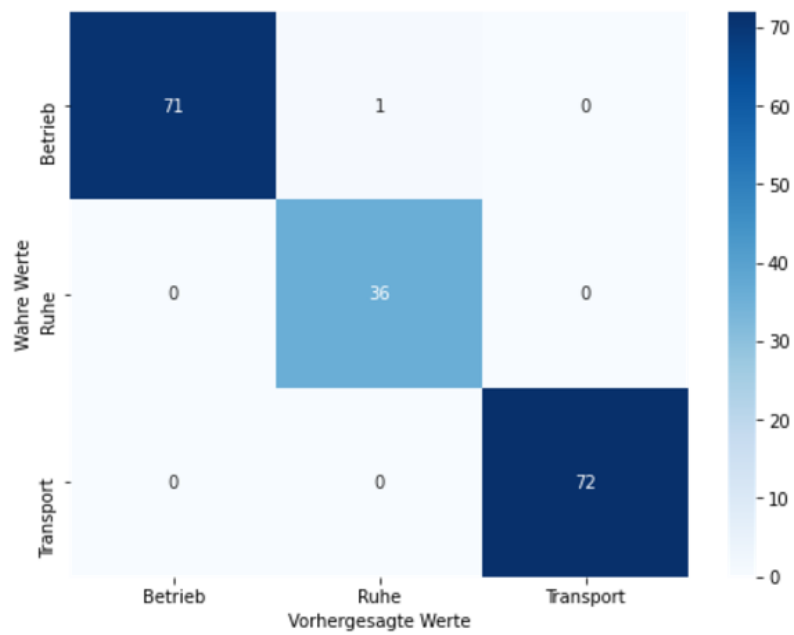


Abbildung 2: Darstellung der Konfusionsmatrix mit Berechnung der Merkmale.

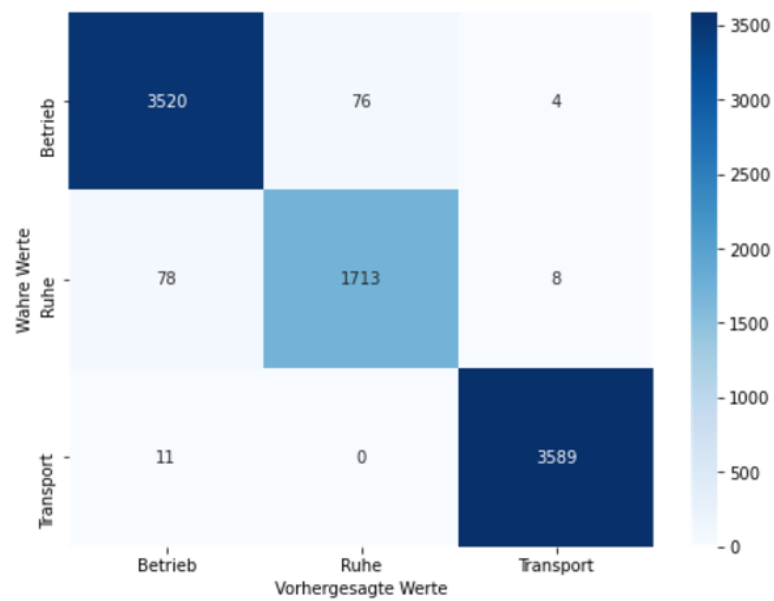


Abbildung 3: Darstellung der Konfusionsmatrix der Rohdaten ohne Merkmalsberechnung.

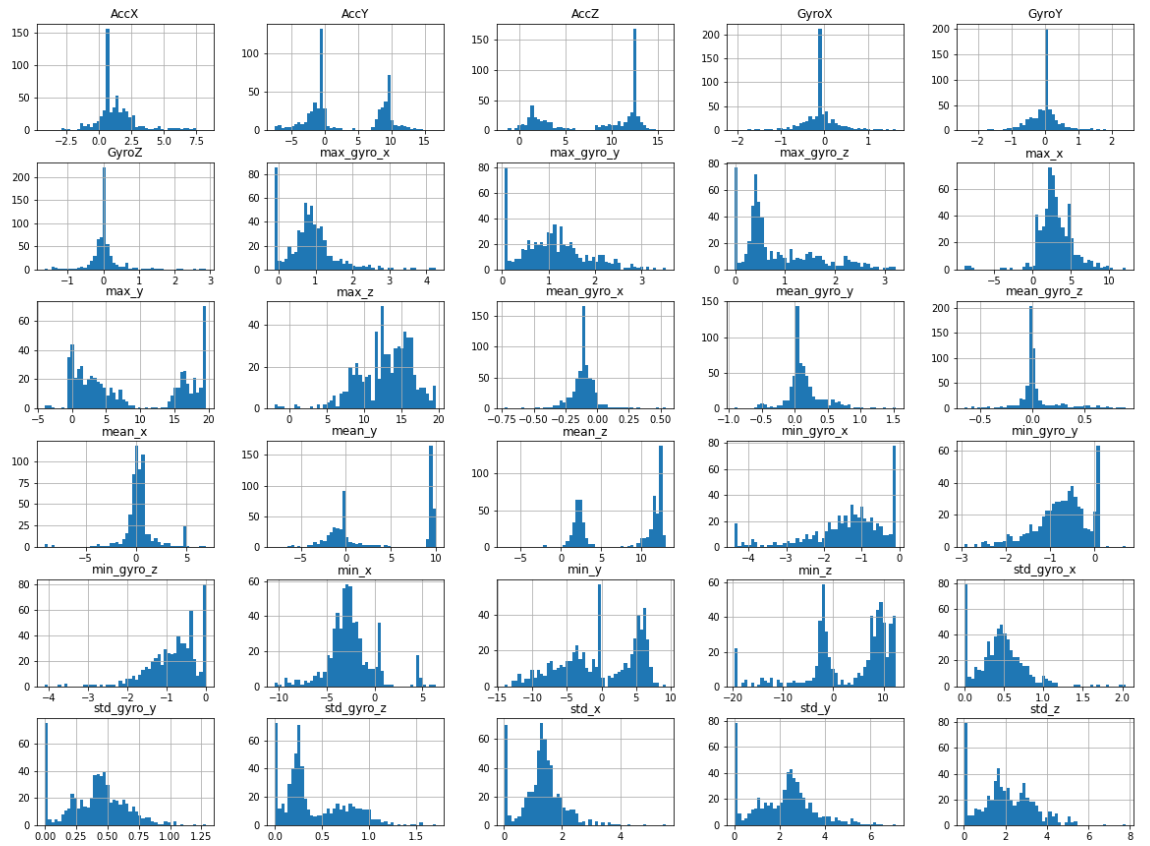


Abbildung 4: Darstellung des Histogramms mit Merkmalen und Rohdaten.

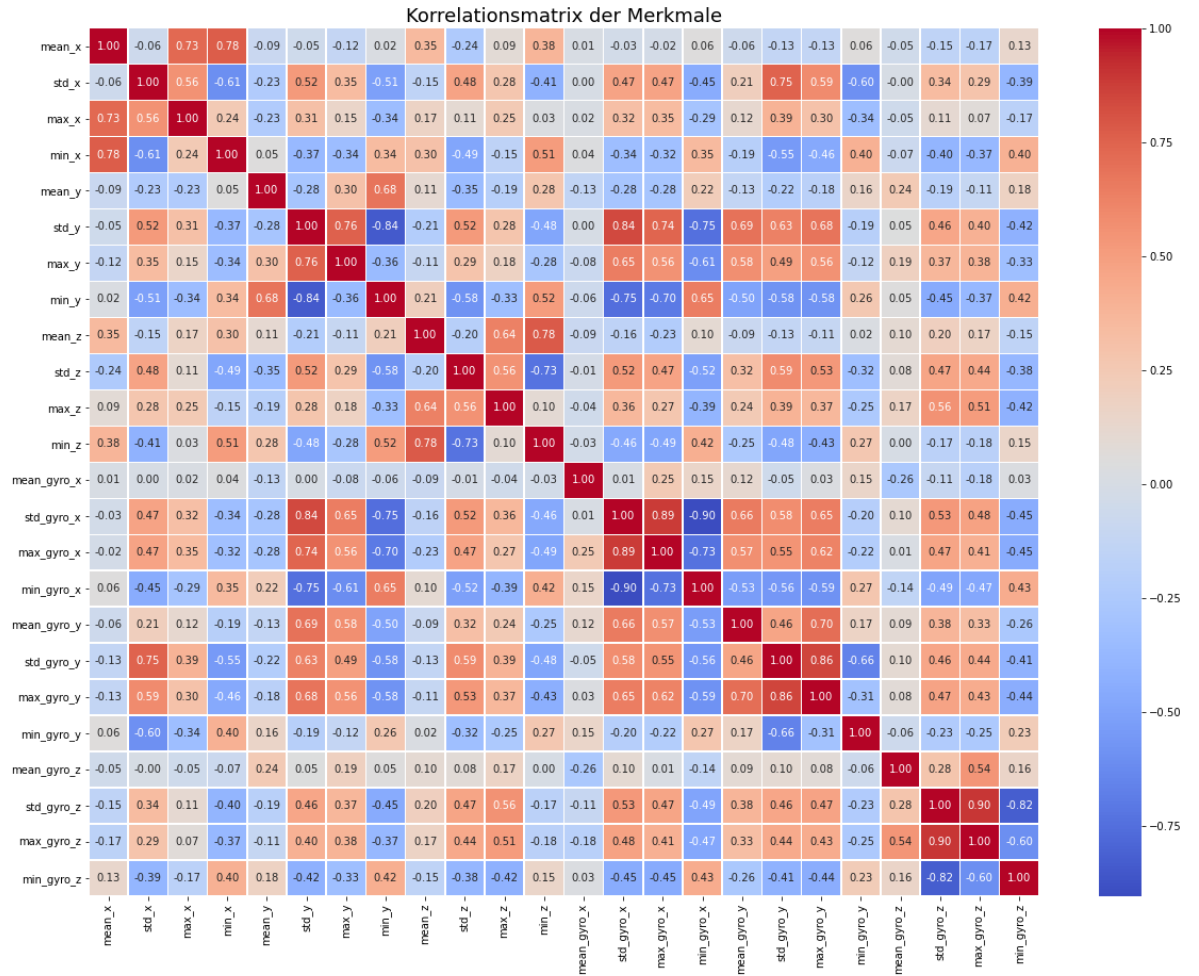


Abbildung 5: Korrelationmatrix der Merkmalen und Rohdaten.

Literatur

- [dat24a] datascientest. Website, 29.1.2024. <https://datascientest.com/en/knn-what-is-the-knn-algorithm>.
- [dat24b] datatab. Website, 29.1.2024. <https://datatab.de/tutorial/pearson-korrelation>.
- [ibm24] ibm. Website, 29.1.2024. [https://www.ibm.com/topics/knn#:~:text=The%20k%2Dnearest%20neighbors%20\(KNN,used%20in%20machine%20learning%20today](https://www.ibm.com/topics/knn#:~:text=The%20k%2Dnearest%20neighbors%20(KNN,used%20in%20machine%20learning%20today).